SOFTWARE METRICS PROJECT

CodeSentinel

Software metrics and multi-model defect prediction

Upload code → measure every function → score its defect risk → explore it in a dashboard



50,826
function snapshots mined

7
ML models compared

12
open-source Python repos

WHY IT MATTERS

Reviewers cannot read everything

Where should limited review time go first?

2.8%

Defects are rare

Only 2.8% of the functions we studied were later changed by a bug fix.

8

Metrics are cheap

Size, complexity and Halstead measures come straight from the source. No history needed.

3.4×

Ranking pays off

Reviewing the riskiest 10% of functions finds about a third of future defect-fixed ones.

THE IDEA

From source code to a risk ranking

01

Ingest

Zip upload, pasted code or the built-in demo

02

Measure

radon and lizard compute metrics per function

03

Predict

Seven models score defect risk

04

Explain

Dashboard, drivers and a report

The parser handles Python, Java, C/C++, JS/TS, Go and C#. The trained models cover Python and C/C++.

STEP 2 · MEASURE

Eight metrics per function

Metric	What it captures
Lines of code	Total size of the function
Executable LOC	Non-blank, non-comment lines
Cyclomatic complexity	Independent paths through the code
Halstead volume	Size of the implementation, in bits
Halstead difficulty	How error-prone it is to write or read
Halstead effort	Mental effort to develop the function
Comment %	Share of lines that are comments
Parameter count	Number of function parameters (Python models)

Halstead measures are token-based: radon's AST version undercounted and returned 0 for many functions.

STEP 3 · TRAIN

Two training datasets

DEFAULT

Python-native

50,826

function snapshots

Mined from the git history of 12 repos

requests, flask, click, jinja, httpx, rich and more

2.8% defective

BENCHMARK

NASA MDP

12,211

C/C++ modules

6 projects: CM1, JM1, KC1, PC1, PC3, PC4

Classic public defect dataset

18.9% defective

STEP 3 · TRAIN

How a function becomes “defective”

1

Snapshot

Take the code at five dates, from 2021-01 to 2023-01

2

Measure

Run the same extractor the app uses on every function

3

Look ahead

Search the next 12 months for bug-fix commits, found by commit-message keywords

4

Label

A function is defective if a bug-fix touched it in that window

Time-forward labels mimic a post-release defect count. A function is identified by its file and qualified name.

STEP 3 · TRAIN

Seven models, one pipeline

Logistic regression
Default · interpretable

Naive Bayes
Fast baseline

Linear SVM
Margin-based

Random forest
Tree ensemble

Gradient boosting
Boosted trees

MLP
Neural network

Stacking ensemble
Combines the others

Isolation Forest
Flags unusual functions

Every model uses log-transform, scaling, class weights and sigmoid calibration, so a displayed 12% risk means about 12%.

STEP 3 · EVALUATE

Tested the way it will be used

Grouped CV by file

No file ever appears in both the training and the test fold, which prevents leakage.

Leave-one-repo-out

Train on 11 repositories and test on the 12th, which mimics a new project.

Temporal split

Train on the first three snapshots and test on the last two.

Reported: ROC AUC, PR-AUC, Recall@10%, F1 and MCC. Accuracy is misleading when only 2.8% of samples are positive.

RESULTS

Model leaderboard

Model	Cross-repo AUC	Temporal AUC	Recall@10%
Logistic regression	0.735	0.739	34%
Naive Bayes	0.735	0.738	36%
Linear SVM	0.734	0.740	34%
Stacking ensemble	0.719	0.745	30%
Gradient boosting	0.718	0.741	30%
MLP	0.718	0.708	26%
Random forest	0.670	0.714	22%

3.4×

more defect-fixed functions than random picking, when reviewing only the riskiest 10%

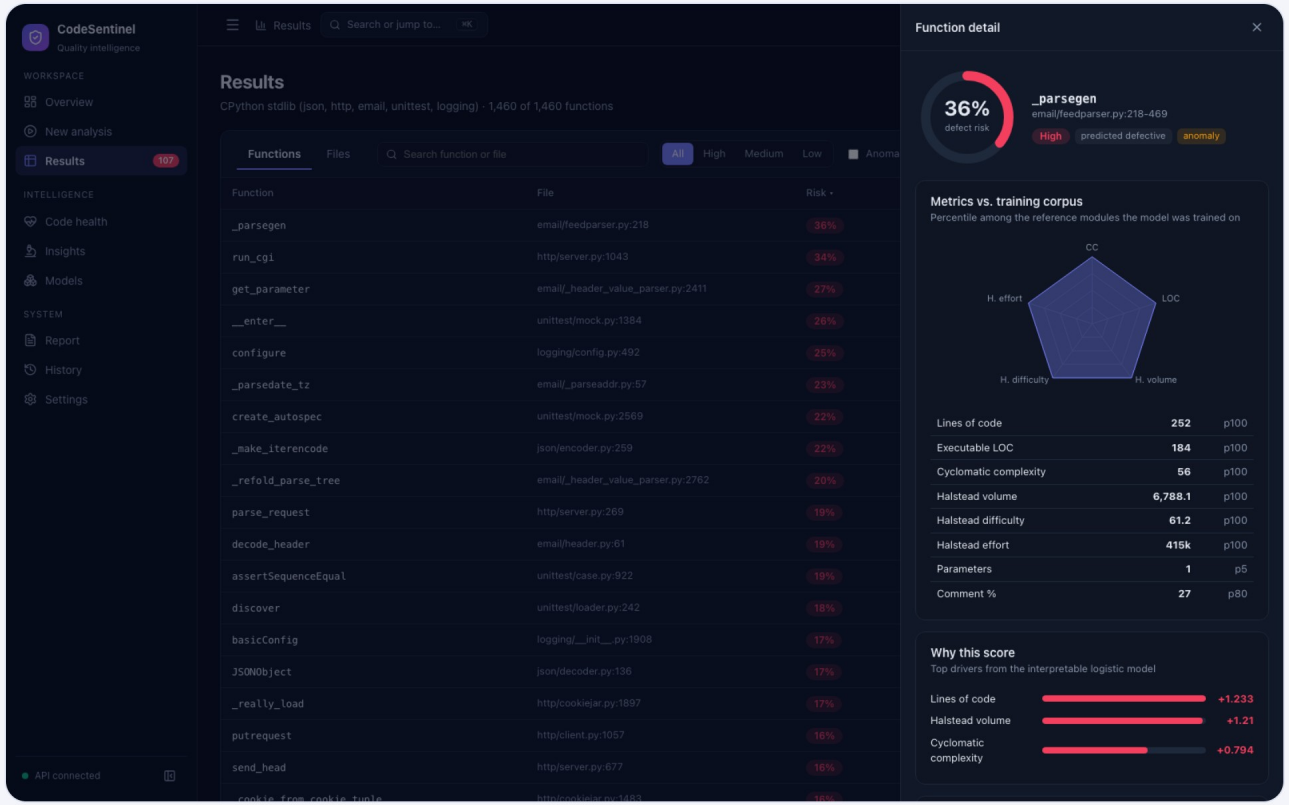
KEY FINDING

Simple models won.

Logistic regression (0.735 AUC) beats random forest (0.670). With only 2.8% positives and metrics-only features, complex models overfit, while logistic regression stays interpretable.

STEP 4 · EXPLAIN

Why is this function risky?



Every score comes with a reason

Gauge: a calibrated defect probability

Drivers: model coefficient × standardised value, such as lines of code +1.23 and Halstead volume +1.21

Radar: percentile versus the training corpus

Advice: rule-based actions, e.g. split into smaller functions

UNDER THE HOOD

Architecture

Frontend

React and Vite

Recharts and lucide icons

Nine pages, light and dark theme

API

FastAPI served by uvicorn

Analyze, compare, models, EDA and runs endpoints

Zip-slip and size guards

ML and storage

scikit-learn models saved with joblib

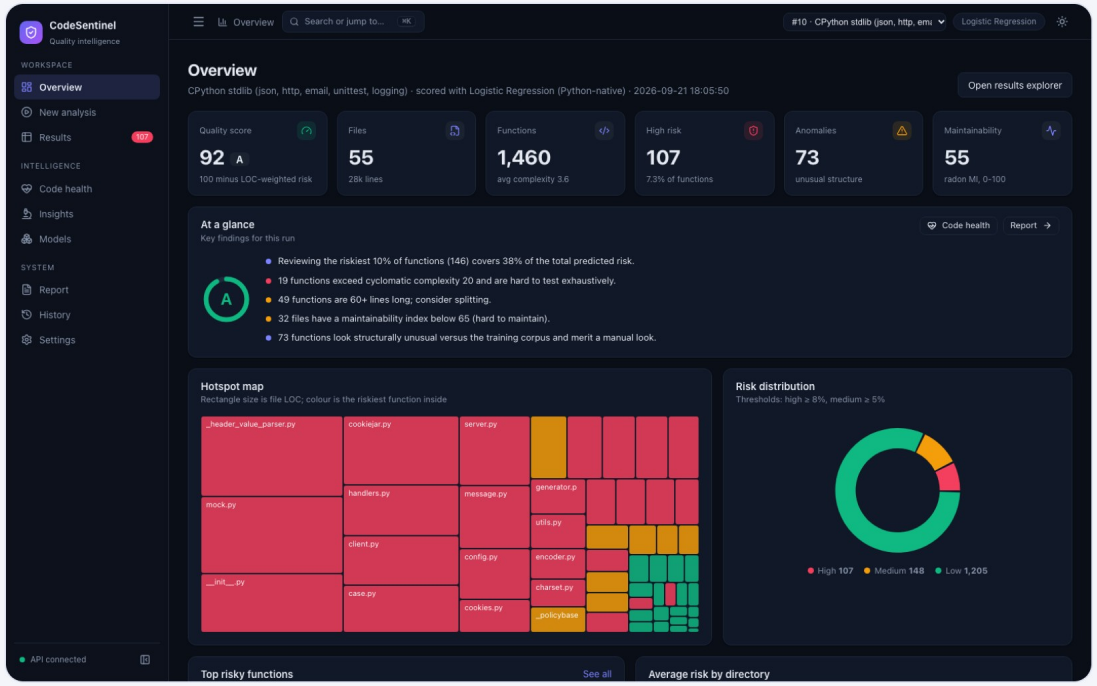
radon and lizard extractors

SQLite run history

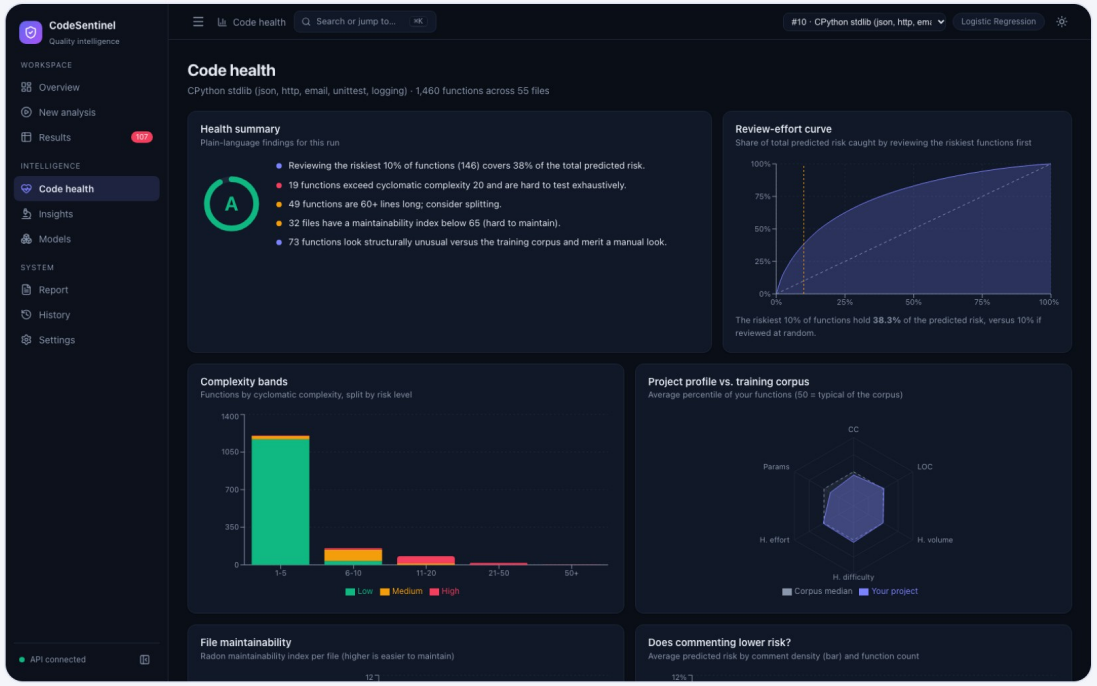
20 automated pytest tests cover the API and the mining code.

PRODUCT TOUR

Overview and Code health



Overview: grade, KPIs and a hotspot map sized by lines of code



Code health: where reviewing first pays off most

PRODUCT TOUR

The rest of the workspace

Results
Search, sort and export every function

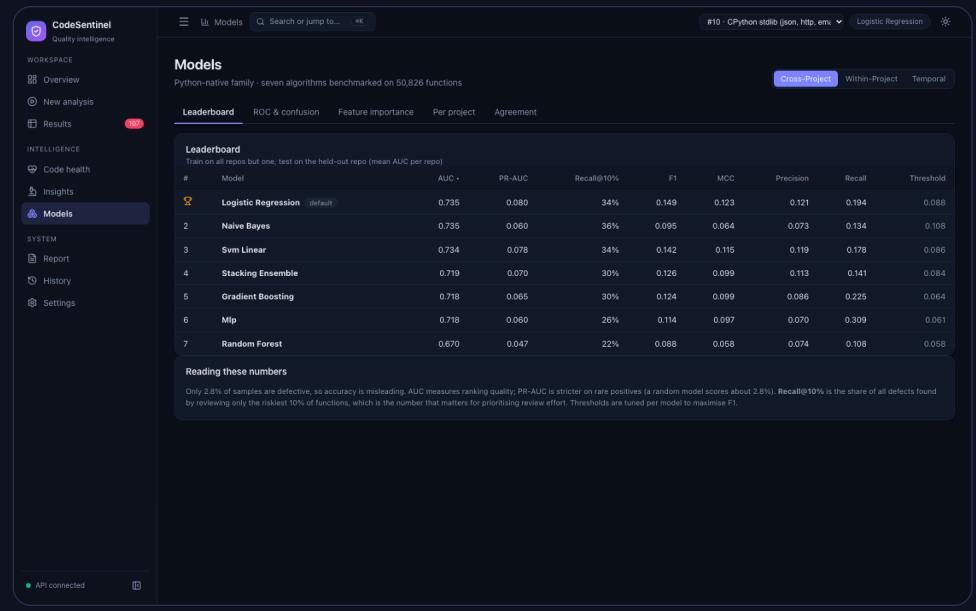
Insights
Defect drivers and distributions

Models
Leaderboard, ROC and agreement

Report
Printable summary with PDF export

History
Reopen, compare and trend runs

Settings
Thresholds, default model, theme



LIVE DEMO

Five steps to show

1. **New analysis → Demo project**
   Scans part of the CPython standard library: 1,460 functions in 55 files

2. **Overview**
   Quality score 92, grade A, with 107 high-risk functions

3. **Results → open _parsegen**
   36% risk, complexity 56, 252 lines, plus the drivers behind the score

4. **Code health**
   The riskiest 10% of functions hold 38% of the predicted risk

5. **Report**
   Copy as Markdown or print to PDF

HONEST LIMITS

Threats to validity

Noisy labels

Commit-message keywords miss silent fixes and include some non-bug “fix” commits.

Narrow sample

12 repos, mostly libraries by experienced maintainers.

Renames

A renamed or moved function is treated as a new function.

Metrics ceiling

Metrics-only features plateau near 0.73 AUC. Prioritise review; do not gate releases.

The NASA family is out of domain for Python and is kept as a benchmark only.

NEXT STEPS

Where the project can go

Add change history

Churn, authors and recency would likely lift accuracy, but need git data at analysis time.

Train on more languages

Mine Java, JS/TS and Go repositories the way the Python set was mined.

Plug into CI

Post the riskiest functions as a comment on each pull request.

TAKEAWAYS

Questions?

- Simple metrics rank functions by defect risk (about 0.74 AUC).
- Small, calibrated and explainable models work best.
- The app covers the whole flow: upload, score, explain, report.